Excerpt — Implementation, §4.1–4.6

Jakub Adam Trzykowski

This chapter builds the backend one construct at a time, checking each against the program presented in *A Guiding Example*. It opens with a short look at VMIR itself, enough that the listings which follow read without a preamble, and then starts from the smallest thing a verifier can do: execute a heapless instruction stream and discharge the obligations it raises — the prover's tiers, and the rewrite rules they run underneath every one of them. The symbolic heap comes next, without conditional structure at first, so that its operations can be stated on their own; then fields, the instructions that move permission; then datatypes, whose rules are what a fold and its unfold cancel by; then predicates, which is where that cancellation is put to use. Domains and the quantified axioms they carry close out the values a program can build, and only then does the chapter turn to calls, method bodies, and control flow — which is what forces two heaps to be reconciled. Wildcard permissions, functions and loops close the chapter, each needing the ones before it and nothing after.

The order is deliberate in one further respect. Each section says what its construct costs the verifier and, where the mechanism differs from Silicon's, where the difference lies. The claim the chapter is accumulating is not that any single mechanism is novel, but that a particular set of them composes into a verifier on which the great majority of a Prusti program's obligations are true by construction rather than by proof — and *Putting It Together* is where that claim is checked against the whole of the guiding example.

4.1 VMIR: The Intermediate IR

The verifier does not execute Viper. It executes *VMIR*, the intermediate representation of *The Verifier's Intermediate Representation*, and every construct of *the guiding example* reaches the verifier only through a lowering. What this section adds is how VMIR *reads*, so that the listings of the sections after it need no preamble of their own. It is deliberately partial — each construct arrives with the piece of VMIR it needs.

What the frontend settles first. Lowering does not start from source text. It starts from a typed abstract syntax tree, and the resolution that produces one is the same Silver [1], [2] performs: the surface language's ambiguities are settled before VMIR is in question. Every intermediate expression carries the type it was inferred to have, and the values Viper lets a program leave implicit — the amount in an `acc(x.f)` written without one, which is `write` — have been filled in. The translator therefore lowers a program in which nothing remains to be inferred, and no lowering rule has to recover what a piece of syntax meant.

Flat and named. VMIR is in static single assignment form, and expressions are flattened: every operation is an instruction of its own, its operands are temporaries rather than expressions, and its result is a fresh temporary with the type written out. Listing 1 is two fragments of the guiding example: the comparison

that decides `account_deposit`'s branch, and the variant-range disjunction out of `own_Transaction` (*the guiding example*), with the repeated discriminant read abbreviated to `d`.

Viper

```
var _tmp0: i32
var d: isize

var b: bool :=
  lt_i32_i32(
    i32_cons(0), _tmp0)

assert d == isize_cons(0) ||
       d == isize_cons(1)
```

VMIR

```
e0: i32 := fresh
e1: isize := fresh

e2: i32 := i32_cons(0)
e3: bool :=
  lt_i32_i32(e2, e0)

e4: isize := isize_cons(0)
e5: Bool := e1 == e4
e6: isize := isize_cons(1)
e7: Bool := e1 == e6
e8: Bool := e5 ? true : e7
assert e8
```

Listing 1: Nesting is flattened and every intermediate value is named.

Two things in that listing are worth naming. A declaration without an initialiser is a `fresh` value and no more: nothing is assumed about it, and there is no store mapping `_tmp0` to anything, only the temporary the name was lowered to. The declaration *with* an initialiser produces no instruction of its own either — `b` is a name for the class `e3` landed in, and an assignment to a source variable rebinds the name rather than building anything.

And the disjunction has become a conditional. VMIR has no negation, conjunction or disjunction of its own: the only boolean connective it has is the ternary `c ? a : b`, and the other three are written in terms of it — `!b` becomes `b ? false : true`, `a && b` becomes `a ? b : false`, and the `||` above becomes `e5 ? true : e7`. One rule then covers what would otherwise need several. That is the shape of every desugaring in the chapter — a surface form is replaced by the one primitive that already had to exist.

A textual representation. VMIR is a language with a syntax rather than just an internal data structure, and the listings in this chapter are written in it. That was a secondary goal rather than a consequence: a program can be printed after lowering, read back, diffed against an expected form, and handed to something other than the verifier this thesis builds. A lowering is then testable on its own terms, and an alternative backend over the same IR — one that discharges the obligations a different way — needs no part of what follows.

VMIR-lite. The real form spells out mechanical detail that a listing does not always need to carry. A listing tagged *VMIR-lite* in its corner drops that detail, so what is left is what the snippet is about. It is the same language, and it agrees with the real form wherever the point being made lives. A listing tagged plain VMIR is what the verifier would actually be given.

4.2 Execution Model

The verifier executes a program symbolically, in Silicon's sense: it walks the instructions in order, maintains a symbolic state describing what is known at each point, and raises an obligation wherever the program claims something. What differs is where that state is kept. Silicon keeps its facts in the solver and asks Z3 what follows from them; here the state is a structure the verifier owns and reads directly. The bookkeeping that buys is the subject of most of this chapter. What it buys back is that a question about the state can be answered by looking rather than by asking, and that the representation can be chosen to make the questions a Prusti encoding actually raises (*What Prusti Demands of a Verifier*) the cheap ones.

What that state is, and what executing an instruction does to it, is best seen on a fragment small enough to walk end to end. Listing 2 is heapless, five statements long, and carries one obligation.

```viper
var a: Int
var b: Int
var goal: Bool := a * 2 == b * 2
assume a == b
assert goal
```

Listing 2: The fragment this section walks: a goal built, and then the equality that settles it assumed.

The obligation is an equality between two terms the program built, and the program builds it before it is given the fact that settles it. Listing 3 is what the verifier is given. Each declaration becomes a value about which nothing is assumed, and each comparison an instruction of its own; the declaration of `goal` becomes no instruction at all, since it is a name for the class the comparison already landed in. What the `assume` and the `assert` name is therefore a temporary rather than an expression.

```vmir
e0: Int := fresh // a
e1: Int := fresh // b
e2: Int := e0 * 2 // a * 2
e3: Int := e1 * 2 // b * 2
e4: Bool := e2 == e3 // goal
e5: Bool := e0 == e1
assume e5
assert e4
```

Listing 3: Viper Listing 2 lowered. Original variable names in the comments.

The verifier executes this line by line, updating the symbolic state at each step, and when it reaches the assertion it checks whether the obligation is satisfied.

Symbolic state. There is no separate store of what the verifier knows. The e-graph *is* the symbolic state, and a VMIR temporary is nothing more than a handle into it: executing an instruction builds the term it names and yields the e-class that term landed in. The walk is forward and single-pass — instructions are executed

in order and none is revisited — so the state at any point is the graph as it then stands, and everything learned up to that point is in it.

Each kind of instruction in Listing 3 does one thing to that graph, and the four kinds it uses are most of the language.

A `fresh` inserts a new e-node into a new e-class. Nothing is said about it and nothing is equal to it, and the temporary being defined becomes the handle on that class — so after the first two lines, `e0` and `e1` name two distinct classes about which the verifier knows nothing at all.

An operation resolves the classes its operands name, and inserts a node over them. `e2` is a `*` node whose two children are the class of `e0` and the class of `2`, and `e2` becomes the handle on the class that node landed in. Insertion is hash-consed: a node with the same operator over the same child classes is not a second node but the one already there, so building a term twice costs a lookup and yields the same class. `e3` is not that case yet — `e0` and `e1` are still distinct classes, so `e1 * 2` is a second node — and `e4` is an `==` node over those two, which nothing so far relates.

An `assume` is a union. `assume e5` merges the class of `e5` with the class of `true`, and that is the whole of how a fact enters the state. One rewrite then fires on what it produced: an `==` node whose class is known `true` unions its two arguments, so `e0` and `e1` become one class.

That union runs backwards into terms already built, and Figure 1 is the graph on either side of it. `e2` and `e3` are applications of `*` over the classes it merged, so congruence closure merges them as well, without either instruction being re-executed and without either term being rebuilt. `e4` is then an `==` node whose two children are one class, and a rewrite folding an equality between a class and itself to `true` is what puts `e4` in the class of `true`. Had the program built the goal *after* the assume instead, hash-consing would have delivered the same state on the way in. Which order the program happens to use is therefore not something the verifier has to care about.

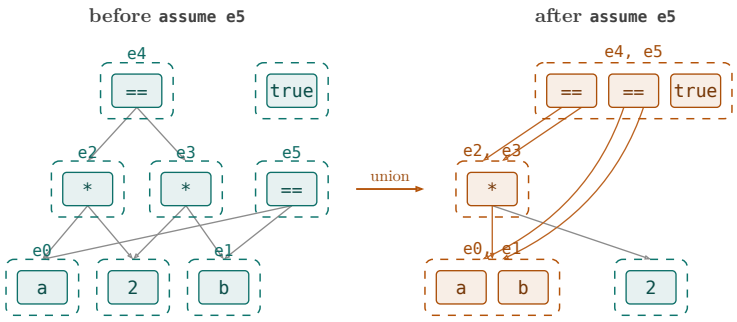

Figure 1: The state either side of `assume e5`. A solid box is an e-node, a dashed box an e-class, an arrow runs from a node to the class of each of its arguments, and the handles a class answers to are written above it. The classes in amber are the ones the `assume` and the rewrites over it change; the class of `2`, which nothing reaches, is drawn as it was on the left.

The assertion on the last line is the question of whether `e4`'s class is the class of `true`, and here it is one lookup. Section 4.3 makes the mechanisms this walk leaned on precise: which rewrites there are, when they are run over the graph, and what the verifier does with an obligation a lookup does not settle.

Well-definedness. Building a term is not always free of obligations. Some operations are partial, and an instruction applying one raises a side condition saying that it was applied where it denotes something. Division is the case in this fragment: `x / d` means nothing where `d` is zero, so `e2: Int := e0 / e1` raises `e1 != 0` as an obligation, at the point the term is built and whatever the surrounding expression goes on to do with the result.

Path conditions. The obligation is not always raised flatly, and a division is the smallest thing that shows why. A program guards one by writing the check into the expression, as in `var y: Int := d != 0 ? x / d : 0` — the divisor is only ever divided by where the guard says it is safe, and a verifier that ignored that would reject a correct program.

The lowering is what carries the guard to the division. An instruction may hold a *path condition*, written in angle brackets before it: a *cube*, meaning a conjunction of boolean values already in the graph, each taken with a polarity. It is attached at lowering time and is exact, so the verifier reads off what is in force rather than working it out.

```
1 e0: Int := fresh // x                                    VMIR
2 e1: Int := fresh // d
3 e2: Bool := e1 != 0
4 e3: Int := <e2> e0 / e1
5 e4: Int := e2 ? e3 : 0 // y
```

Listing 4: A guarded division. The obligation the division raises is carried under the guard the program wrote.

The `<e2>` on line four is the path condition, and it is one literal: the class of `e2`, taken positively. Without it the division would raise `e1 != 0` outright, which is not provable here and would reject the program for a division it never performs.

The translator maintains that cube as a stack of literals while it walks. Lowering the then-arm of a ternary pushes the condition positively and lowering the else-arm pushes it negatively, each around the lowering of that arm and popped after it, so the stack always holds exactly the literals in force and an instruction takes its cube by snapshotting it.

Not every instruction takes one. An instruction that only builds a term needs no cube, since a term is the same term wherever it was built: the arithmetic, the comparisons and the ternary itself are emitted flat however deeply nested in conditions they stand, and one temporary stands for the term across all of them. An instruction with an obligation is the case that needs the cube, and in the heapless pure fragment that is division and modulo alone. Heap instructions and calls fall on the same line and are taken up where those constructs are (Section 4.6, *Functions*).

What the cube then does is weaken the obligation. One raised under a cube need only hold where the cube does, so what has to be proven is not the goal but the implication pc $\Rightarrow$ goal — here $e_2 \Rightarrow e_1 \neq 0$, whose consequent is the literal the cube already contains.

One state, not two. Carrying the condition on the instruction is what lets the verifier keep a single graph where a forking symbolic execution keeps one state per path. Silicon takes that other route, its `branch` rule taking one continuation per outcome:

> `branch` enables splitting the symbolic execution into two paths: one path (Q_v) is taken under the assumption that v is true, the second path ($Q_{\neg v}$) is taken under the assumption that v is false.
>
> — [3, Section 3.2]

Forking gives each path a state of its own. A fact obtained on one path is simply not present on the other, and every fact a path does hold is unconditional: nothing proven there has to mention the condition the path was taken under. Two paths that share nothing can moreover be explored on two verifiers at once, which Silicon can be asked to do.

A single graph gives sharing instead. Every term built before the condition serves both readings of it, and nothing is copied when the condition arises. The price is that a fact holding on one side only cannot be stated outright, and that an obligation raised under a cube is an implication rather than a goal.

Which of the two pays depends on the obligations. This design assumes, on the evidence of *What Prusti Demands of a Verifier*, that most of them follow from the structure the program has already built, and so are discharged without the cube being assumed at all. What becomes of the ones that are not is Section 4.3.

4.3 Discharging Obligations

Every instruction of the preceding section either produced a value, updated the state, or raised an obligation, and the obligations were discharged only non-specifically. This section describes the mechanism precisely.

The tiers. An obligation is a goal e-class and an optional path condition — an obligation raised where nothing is assumed carries none, and the tiers that exist to handle a condition have nothing to do on it — and the verifier answers it by escalating through named tiers, cheapest first. Each tier is strictly more work than the last, so the ordering is the design: an obligation should cost what it takes to answer it and not what it would take to answer the hardest one. Where the obligations of a real program actually stop is a measurement, and *Results* is where it is made.

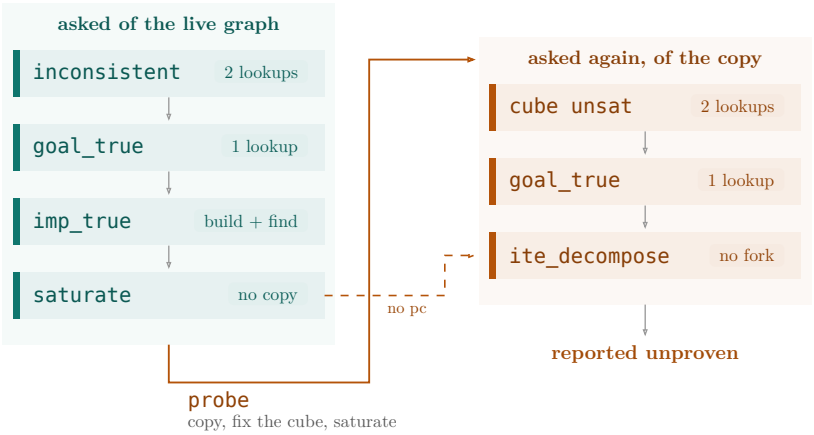

Figure 2: The prove ladder. Each arrow is a tier failing to answer: a tier that does answer discharges the obligation and the search stops there. `probe` answers nothing itself — it copies the graph, fixes the cube in the copy and saturates it, after which the same cheap questions are put to that copy, with `ite_decompose` the last resort on it. The dashed edge is an obligation with nothing to assume: `probe` would change nothing, so the copy's cheap questions are skipped with it.

`inconsistent`. Asks, on a best-effort basis, whether the state contains a contradiction: not every inconsistency the graph implies is found, but every one reported is real. It comes first because a contradictory state proves everything, making all work below it redundant — and it is cheap because a contradiction is recorded as it happens rather than searched for, so asking is two comparisons of class identifiers.

`goal_true`. Asks whether the goal holds on its own, before the implication pc ⇒ goal is built. Building that implication inserts permanent nodes; an obligation with no path condition, settled by constant folding the moment it is raised, should not pay for a term nothing ever reads again.

`imp_true`. Builds the implication and asks whether *it* is `true`. Nothing is cached: discharging an obligation merges its implication with `true` like any other fact of the graph. The tier pays off on repetition — a Prusti encoding raises the same framing obligation at each statement of a run that holds the same permission (*What Prusti Demands of a Verifier*), and later occurrences are answered here for the price of a comparison.

`saturate`. Runs the rewrite rules to a fixpoint and asks again. It is the first tier that does work rather than a lookup, and it sits above the copying tier because that work is not thrown away: saturation runs on the live graph, so every equality it derives stays for every later obligation. The rules (*The rewrites*) are identities, so this tier answers obligations true by rewriting rather than reasoning — a boolean connective collapsing to the condition it was built from, an assumed equality carried into congruence — none of which needs a path condition.

`probe`. Assumes the path condition. A cube's literals hold on one path only, so they cannot be merged into the live graph without the verifier believing them everywhere; instead the graph is copied, the literals fixed in the copy, and the copy saturated. Inside a method body the copy is not per obligation: a block's cube is

static, so one scratch graph serves every obligation the block raises (*Control Flow*). The tier earns its cost by what would otherwise be unprovable — a guarded division, or any obligation raised under a condition the program itself wrote — rather than by how often it fires.

An obligation with an empty path condition, or whose literals the live graph already knows at the polarity the cube wants, passes this tier for free — the copy's cheap questions cannot answer differently from the live graph's — and drops straight to the last tier.

`ite_decompose`. The last tier, and the narrowest by design. VMIR spells $c \Rightarrow e$ as $c\,?\,e : \text{true}$ (Section 4.1), so a goal with a `true` arm is an implication written as a conditional; the tier reads it back as one — assume c, re-check e, telescoping a chain of nested implications one assumption at a time. Its conditions are *read off the goal* rather than searched for, which bounds it and separates it from a case split. It works whichever arm carries `true` — the antecedent it assumes just changes polarity — so it covers an obligation the program wrote as an implication without needing to tell the two shapes apart, and no tier above can do anything with a conditional as it stands.

Constant folding. Underneath every tier is one rule set, always on. Congruence and hash-consing come from the e-graph itself; the rest is an analysis and a set of rewrites.

The analysis is constant folding, attached to every e-class. A class whose term evaluates to a literal carries that literal, and it propagates upwards: the operators fold when both operands are known, an integer cast folds a known integer, and a conditional whose condition is known takes on the data of the arm that condition selects. One case does more than evaluate: a class forced to carry two different literals is a contradiction, which is what the first tier looks for.

The rewrites. The rules that fire during saturation are identities rather than a theory, and deliberately incomplete: an "explosive" rule that grows the graph — one whose right-hand side names a term the left-hand side did not already mention — is excluded on principle, no matter how useful, because its cost is paid by every saturation from then on and not just the obligations it helps. Almost every rule below is instead a *subset* rewrite: the right-hand side spells out nodes the left-hand side's e-class already contains, so applying it adds no e-node, only a union. Table 1 lists them.

$x + 0 \to x, \quad x - 0 \to x$	units
$x \cdot 1 \to x, \quad x/1 \to x, \quad x \cdot 0 \to 0$	units and annihilation
$(x - p) + p \to x, \quad (x + p) - p \to x$	an amount taken and given back
$x - x \to 0$	an amount taken in full
$x = x \to \text{true}, \quad x < x \to \text{false}$	reflexivity
$(b = \text{true}) \to b, \quad (b = \text{false}) \to b \,?\, \text{false} : \text{true}$	boolean equality is a ternary
$\text{true} \,?\, x : y \to x, \quad \text{false} \,?\, x : y \to y$	a folded condition
$c \,?\, x : x \to x$	both arms agree
$c \,?\, \text{true} : \text{false} \to c$	the condition, literalized in both arms
$c \,?\, c : \text{false} \to c, \quad c \,?\, \text{true} : c \to c$	the condition, recurring as an arm
$c \,?\, c : \text{true} \to \text{true}, \quad c \,?\, \text{false} : c \to \text{false}$	the condition, recurring as its other arm

Table 1: The rewrites: arithmetic identities, chosen for the shapes a verified program's accounting produces, and conditional identities over `ternary`, the encoding of every boolean connective (Section 4.1).

The first pair of conditional rules is the one the rest of the chapter relies on: once a condition is known, everything built under it reduces by the same rule that simplifies any other conditional, with no case analysis anywhere.

One more rule turns a proven equality back into a merge: an `==` node whose class is known `true` unions its two arguments. That is what lets an assumed equality participate in congruence rather than sitting inertly as a fact, and it is the rule doing the work in Listing 3. A disproven `==` node is read back the other way, into whatever distinctness can be derived from it — that family is where *ADTs* picks up.

What the ladder cannot do. Three things are missing from it, and the first two are the price of the property the rest of the section was arguing for: every tier answers with work proportional to the obligation in front of it, and a tier that searches does not.

There is no decision procedure for arithmetic. Rewriting closes the identities of the table above and nothing beyond them, so an obligation like $i < n \vdash i + 1 \leq n$ is reported unproven.

There is no case split. A goal that needs genuine reasoning by cases over an opaque condition is reported unproven rather than forked on. Even in its simplest shape, a case split means probing the graph once per branch, on every obligation that reaches it — explosive in a way `ite_decompose` is not, since that tier reads its conditions off the goal instead of choosing them. Whether a cheaper form exists is a question this design leaves open rather than settles.

There is no disequality store. An e-graph records that two terms are equal and has no way to record that they are not, so distinctness is derived where it can be — from a constant-folding collision, from the contrapositive of congruence, or from an observation that separates two values — and is otherwise deferred (*ADTs*).

4.4 The Symbolic Heap

Everything so far has been values. The heap is the other half of the state, and this section is what it is made of: what a permission-carrying thing is, where a chunk holding permission to one lives, and what happens when two chunks turn

out to be at the same place. The instructions that move permission around are Section 4.6; the two declaration forms that produce something to hold permission to are Section 4.5 and *Predicates*.

The section is deliberately restricted to the *straight-line* heap. Everything conditional is deferred to *Control Flow*, which is where a branch first forces two heaps to be reconciled and so where conditional chunks are actually motivated; what a chunk is and what partitioning it buys can be settled without any of that.

Locations. Viper distinguishes between the two kinds of resource a program can hold permission to: a field of an object, and an instance of a predicate. VMIR collapses that distinction and knows only about *locations*.

A location type is written `&[g] T @ p` and has three components. The *stored type* `T` is the type of the value held there. The *permission bound* `p` caps how much permission any single location of this kind may hold, and is either a rational constant or `*`, meaning unbounded. A Viper program produces only the two extremes: `1/1` for a field, matching `write`, and `*` for a predicate instance, since nothing stops a program from holding arbitrarily many of those. The *group* `g` identifies the declaration the location came from.

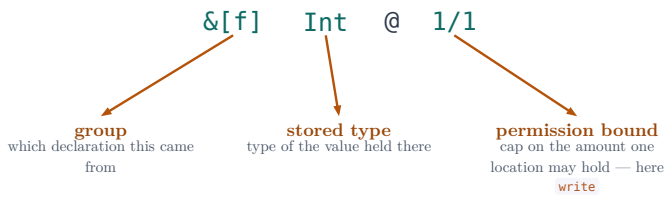

Figure 3: Anatomy of a location type, against the concrete instance `&[f] Int @ 1/1` rather than the schematic `&[g] T @ p`.

Because all three live in the type rather than in a side table keyed by syntax, a location describes itself. This matters because locations are ordinary values and can therefore be *computed*: a location may be produced by a ternary or returned from a call. However it was produced, the verifier can still recover what is stored there and how much permission it may carry, and nothing downstream requires it to have come from a syntactic field access.

Chunks. What the heap holds is a set of *chunks*. A chunk is a triple: the *location* it sits at, the *permission amount* held there, and the *value* stored. For a field chunk the value is the field's contents; for a predicate it is the instance's snapshot (*Predicates*).

A chunk names its parts by e-class rather than by term. In particular its location is an e-class of the graph that already carries the program's equalities, so two chunks sit at the same location exactly when their location e-classes are one, and establishing that costs a lookup rather than a query.

The triple is the whole of it on a straight-line path. A chunk carries two further components that mean nothing until later: a reachability guard, which records the branch a conditionally-held chunk survived (*Control Flow*), and a provenance for

its value, which lets a function's body be replayed at a call site (*Functions*). Both are introduced where they do something.

Partitioning. The symbolic heap is not a single flat map from locations to chunks. It is partitioned by *location kind*, which is nothing other than the location type itself: group, stored type and permission bound. Every chunk lives in exactly the partition its type names, and the two are looked up together: an access resolves first to a partition and only then to a chunk within it.

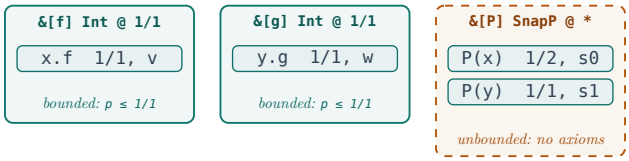

Figure 4: The symbolic heap as boxes, one per location kind. A chunk lives in exactly one box; `x.f` and `y.g` are in different boxes despite sharing a stored type and bound; the predicate box is the `*`-bounded one, dashed, and receives no location axioms.

Since the kind is read off the location's type, a computed location is filed as reliably as one written as a field access, and locations from different declarations never meet. Aliasing is therefore only ever a question *within* a partition, where identity is the location's e-class: two accesses alias exactly when their location terms have been merged. What remains is to say what becomes of a pair of chunks that does turn out to be at one location, and what is assumed of a pair that does not.

Consolidation. Equality reasoning can therefore bring two chunks of a partition to one location after both are already held, simply by merging the e-classes of their locations. The two then hold permission to the same thing, and their amounts have to be added up before either can be counted against a demand. The verifier does this at lookup: resolving a location scans the chunks of that location's partition, compares canonical e-classes, and folds together any that have collapsed into one. Because the heap is indexed by kind, that scan is linear in the size of one partition and never touches another.

The permission is the easy half of a fold: the amounts add, and the surviving chunk holds $p_0 + p_1$. The value takes more care. The two chunks may disagree about what is stored, and only one holding positive permission has any claim to be right about it. The merged value is therefore picked asymmetrically, as $p_0 > 0 \,?\, v_0 : v_1$, and the agreement of the two is *assumed* rather than asserted:

$$p_0 > 0 \wedge p_1 > 0 \Rightarrow v_0 = v_1$$

Silicon's `combineSnapshots` splits the same three ways and, where neither fraction is definitely positive, creates a *fresh* snapshot constrained by both implications — on the stated grounds that using either of the two values and constraining it would be unsound. The asymmetric pick is that judgement made total rather than contradicted: the value is case-analysed on $p_0 > 0$ instead of being left to a new

symbol, which needs no fresh name and is strictly more precise, since on each side of the case the value that is genuinely held is the one selected.

Stating it this way makes both cases come out on their own. When both fractions are positive the antecedent holds, saturation collapses the implication to $v_0 = v_1$, the two values are merged into one e-class, and the asymmetry of the pick disappears with them. When one fraction is zero the antecedent is false, nothing is assumed of a value nobody holds, and the pick selects the one genuinely held. Equating the values outright would be unsound in the second case, and is unnecessary in the first.

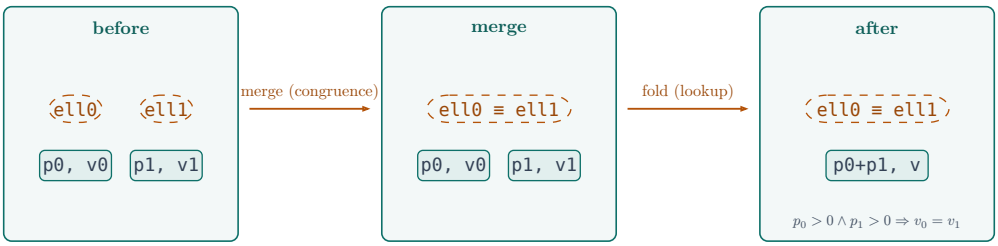

Figure 5: Consolidation. The merge of two location e-classes is what **triggers** the fold that follows it, rather than being the fold: a separate step, drawn as a separate arrow.

Silicon carries the same obligation in a different shape. A chunk there holds terms, so whether two chunks describe the same location is a question for the solver, and the *state consolidation* that answers it iterates to a fix-point: merging one pair contributes equalities that may in turn permit the next merge. Its worst case is cubic in the number of heap chunks, and the optimisations carried by the implementation are reported not to change that bound [3, Section 3.4.2]. Consolidation is therefore scheduled rather than continuous — partially when permissions are added, fully when an assertion fails — which trades completeness against cost.

Holding locations as e-classes removes the fix-point rather than the work. Congruence closure has already propagated the equalities before the heap is consulted, so one pass over one partition suffices and there is no second round to run; and because a partition is indexed by location kind, that pass is linear in one partition rather than in the heap. What that buys is that consolidation can happen at every lookup rather than at chosen points, so the scheduling question does not arise.

Location axioms. Two chunks of one partition that have *not* been merged are the converse case, and partitioning says just as precisely where a non-aliasing constraint has to be injected: between exactly those pairs, since they are the only ones that could still turn out to denote the same location. Both of the axioms the verifier states over a partition come from its bound.

The first applies to each chunk on its own, and is what makes the bound mean anything: a chunk of a kind bounded by b, holding an amount p, satisfies

$$p \leq b$$

Where an amount folds to more than b this leaves the state inconsistent, and the verifier reads that as it should — the path cannot be reached, so anything asked of

it holds. Silicon does the same with `assume-valid-permissions`, constraining each field chunk to at most `write` and concluding from that the infeasibility of certain paths [3, Section 3.4.2].

The second is the non-aliasing constraint, and applies to a pair. For two chunks of the partition holding p_0 and p_1 at locations ℓ_0 and ℓ_1, the verifier states

$$\ell_0 = \ell_1 \Rightarrow p_0 + p_1 \leq b$$

pairwise within a partition and nowhere else. Neither axiom has anything to say where the bound is `*`: with no b to state them against, a partition of unbounded kind receives neither.

Silicon states the same implication, but over receivers and against a fixed bound: for each unordered pair of field chunks it adds the path condition $x = y \Rightarrow p + q \leq 1$ [3, Section 4.6]. Phrasing it over locations instead is what makes it uniform. A field's location function is unary and a predicate's takes the predicate's arguments, but the constraint mentions neither, so one form covers a field, a predicate given a bound, and any other producer whatever its arity — and the bound it is stated against is the one in the location's type rather than `write`.

Partitioning by the whole location type, rather than by the group alone, is what makes this convenient to state. Every chunk of a partition shares one bound, so wherever a pair is considered the b to state the constraint against is already to hand.

It is likewise what makes the bound worth carrying in the location type rather than deriving it from which of Viper's two resource kinds is in play. A frontend that knows a particular predicate is sound to hold at most twice can declare it at `2/1`, and the declaration is not merely documentation: it moves that predicate's partition from the case that yields no constraint to the case that yields one. A bound a frontend can justify is non-aliasing reasoning the verifier then gets for free.

One question the axiom leaves open is how a program ever concludes `x != y` from it, since it is stated over locations and never mentions a receiver. The answer is that it does not have to. Under `acc(x.f) && acc(y.f)` the two amounts sum past the bound, so the axiom's implication forces its antecedent false and the two *locations* are known distinct. Narrowing that to the receivers is then the contrapositive of congruence: from a disproven `f(x) == f(y)`, with every argument pair but one already merged — here there is only one pair — the remaining pair must differ. That step is sound for any function, injective or not, which is why the field's location function needed no injectivity axiom to make it work.

Permission arithmetic. The rewrite rules of *The rewrites* were stated there neutrally, because permissions did not exist yet at that point in the chapter. This is what half of them are for. $(x - p) + p \to x$ is an amount taken by an exhale and given back by the matching inhale; $x - x \to 0$ is an amount taken in full; $x + 0 \to x$ is a give-back against a chunk that was already empty; and $b\,?\,p : 0 \to p$ under an assumed guard is a conditionally-held chunk on the path where the guard holds. Each has a rational form as well as an integer one, and it is the rational form that runs: a permission amount is a rational term, and the accounting a verified program

does with it is almost entirely of these four shapes. An obligation that reduces by them is one the prover answers at its cheapest tier rather than by reasoning about rationals at all.

4.5 Fields

The first of Viper's two ways of declaring something a program can hold permission to is also the smaller one. A field declaration is lowered to a unary function from a receiver to the location of that receiver's field. Prusti emits one field per *primitive type* and never per Rust field, so one declaration, below, is where every value of that type in the whole encoding is stored.

Viper	VMIR
`field f: Int`	`function f(e0: Ref)` `  : &[f] Int @ 1/1`

Listing 5: A field declaration becomes a location function.

The declaration fixes all three components of the location type (Section 4.4): the stored type is the field's own, the bound is `1/1` matching `write`, and the group is the field's name. The last of these is the load-bearing one. A group per declaration is a partition per declaration, so chunks of two distinct fields can never be brought to one location and nothing has to be stated to rule it out. In a Prusti encoding that partitioning is coarser than it looks — every Rust value of that primitive type, anywhere in the program, shares one partition — which is why the aliasing question below is one the design has to answer rather than one the frontend arranges away.

The mapping is not injective. The function is declared and nothing else: no body, no axiom, no postcondition. This is where our verifier diverges from field semantics in Silicon or Carbon. The other two verifiers make fields an injective mapping implicitly. It comes down to how they state the non-aliasing axiom: they talk directly about the receivers and not about the locations.

Does the missing injectivity matter? Not in the usual direction, which is concluding that two receivers are distinct.

```
1 inhale acc(x.f, write) && acc(y.f, write)                    Viper
2 assert x != y
```

Listing 6: Distinctness of receivers, derived without injectivity.

The pair axiom of Section 4.4 gives `f(x) == f(y) ==> write + write <= write`, whose right-hand side constant-folds to false, so the two locations are distinct. The contrapositive of congruence — from `g(a) != g(b)` conclude `a != b` — narrows that to `x != y`. Neither step needs the location function to be injective, which is why this direction is not missed.

Injectivity is the converse: concluding `x == y` from an equality between their locations. A program reaches for it when it learns an aliasing fact by *counting permission* rather than by being told it.

```
1 inhale acc(x.f, 1/2) && acc(y.f, 1/2)                        Viper
```

```
assume perm(x.f) == write
assert x == y
```

Listing 7: The shape that needs an injective field mapping.

The assertion holds in Viper: what is genuinely held at `x`'s location is $1/2 + \ell_x = \ell_y \,?\, 1/2 : 0$, so the assumption can hold only where the condition does. Our verifier rejects it, and injectivity is only the second of two missing steps — concluding that the two locations coincide from the sum reaching `1/1` is arithmetic driving a case analysis on a condition the program never mentions, which is exactly the general-prover reasoning the core is not built to do (Section 4.3).

Were the fact wanted, it could be stated by declaring an inverse beside the location function.

```
function f(e0: Ref): &[f] T @ 1/1                                VMIR-lite
  ensures e0 == f_inv(result)

function f_inv(e0: &[f] T @ 1/1): Ref
```

Listing 8: How injectivity of a field mapping would be stated.

From `f(x) == f(y)`, congruence gives `f_inv(f(x)) == f_inv(f(y))`, and the post-condition rewrites each side to its own receiver.

The case for stating it is stronger in VMIR than in Viper. A Viper program can name a location only by writing a field access, so a receiver is always at hand; VMIR locations are ordinary values (Section 4.4) that can be computed, passed and compared directly, and a frontend reasoning about them that way has no other route back to the receivers. No such frontend is in view here. Prusti's encoding never asks for two references to be equated, let alone by counting permission, so no obligation in the corpus of *What Prusti Demands of a Verifier* depends on a field mapping being injective, and the declaration stays bare.

That is the whole of what a field *is*. What a program does with one — take permission to it, read it, write it — is not field-specific at all, and is the subject of Section 4.6; predicates, which need those operations to state their bodies with, follow in *Predicates*.

4.6 Interacting with the Heap

These four instructions are the hot path: `exhale`, `unfold` and `fold` are among the most frequent statements a Prusti encoding contains, and every `acc` assertion any of them carries is what the operations below ultimately do to the heap. Whatever else the verifier is good at, it has to be good at this.

A program that owns a field takes permission to it, writes through it, reads back what it wrote, and gives permission away again. The following four lines do all of that, over the field Section 4.5 lowered.

```
inhale acc(x.f, 1/3) && acc(x.f, 2/3)                                Viper
x.f := 42
assert x.f > 10
```

```
4 exhale acc(x.f, 1/2)
```

Listing 9: Taking permission to one field in two pieces, writing it, reading it back, and giving half of it away.

The fractions are chosen deliberately: a Prusti encoding never states an `acc` at anything but `write`, so a listing built from one would exercise the arithmetic at exactly one value and hide what the operations do. They are split here for that reason and for no other.

The four statements are taken one at a time below, each becoming one kind of instruction. The listings are VMIR-lite in two respects: the receivers keep their source names rather than becoming numbered values, and the computation of an address is written where it is used rather than on a line of its own.

Taking permission. The heap is not a structure the verifier mutates in place. It is named by a temporary of its own sort — the `h` temporaries, distinct from the `e` temporaries that name e-classes — and an instruction either produces one or consults one. Each conjunct of an assertion is applied on its own, threading one heap into the next, so the `inhale` of Listing 9 becomes one add per `acc`:

Viper	VMIR-lite

`inhale acc(x.f, 1/3)`
`  && acc(x.f, 2/3)`
	`h0 := empty + f(x) @ 1/3`
	`  with fresh`
	`h1 := h0 + f(x) @ 2/3`
	`  with fresh`

Listing 10: An `inhale` becomes one add per `acc`, threading the heap.

Both instructions of Listing 10 are *heap-producing*: each takes a heap and yields a new one, written with the new heap on the left. The chain starts at `empty`, the heap that holds nothing, since this is the first statement of a body. The location is `f(x)`, the location function of Section 4.5 applied to the receiver, and that is the whole of what `x.f` means in this position; elsewhere it means the value stored there, which has to be read out of a particular heap.

The trailing `with` is the instruction's *bind point*: where the value of a chunk it creates comes from. An add is the one direction that has no value of its own — it puts permission somewhere the program may not have held any — so the value is a parameter of the instruction rather than something the verifier decides. `with fresh` is the havoc form, and it is what an `inhale` in a method body means: whatever is at that location, nothing is assumed about it. The other two forms bind the value to a term already in scope, and they are what make a resource body and a fold mean what they mean (*Predicates*). There is no default: a produce that wants havoc says so, which keeps the one case with consequences from being the silent one.

What an add does with the bind depends on whether the partition already holds a chunk at the location. If it does not, a new chunk is created holding the value the bind supplies — for `fresh`, a new e-class with nothing assumed about it. If it does, the two are the same chunk: the amounts add, the value stays as it was, and the bind is not consulted. That is the second add of Listing 10. The first created a

value and gave it `1/3` the second finds `f(x)` already present in the partition, leaves the value alone, and raises the amount to `1/3 + 2/3`, which folds to `1/1`. Two conjuncts naming one location produce one chunk, not two, which is what makes the held amount at a location a single term to compare against. An add can never fail for want of permission.

What it can fail on is the amount itself. Every `acc(l, p)` carries the side condition

$$p \geq 0$$

whichever direction it is applied in, since an assertion naming a negative fraction denotes nothing, and the verifier raises it as an obligation before the add or subtract it belongs to. The amounts of Listing 10 are literals and answer it on the spot; one computed from program values does not, and it is checked like anything else (Section 4.3). Adding permission is thereby unconditional in the heap and conditional only on its own assertion being meaningful.

The merge happens here only because the two locations are syntactically the same and so land in the same e-class immediately. Had the assertion been `acc(x.f, 1/3) && acc(y.f, 2/3)`, the add would have produced two chunks in the partition, and they would still merge later if `x == y` were derived — the e-classes collapse, and the fold that adds the amounts and reconciles the two values under the agreement axiom is *Consolidation* of Section 4.4. Nothing an add does depends on knowing the aliasing up front; it is a lookup that happens to succeed early in this example.

Writing. An assignment produces a heap in which one location holds a new value and every other chunk is untouched:

Viper	VMIR-lite
`x.f := 42`	`h2 := h1 assign(f(x), 42)`

Listing 11: An assignment replaces one chunk's value and frames the rest.

Executing it takes three steps. The location is *resolved* first: `f(x)` is taken to its canonical e-class, the partition of `h1` that this location belongs to is scanned for chunks at that e-class, and any that have since collapsed into one are folded together (*Consolidation*). If nothing answers, the amount held is zero and the write fails for want of permission. Second, the amount of the chunk found is checked against the bound b of that partition — `1/1` for field-originated locations, which is why a program holding a half cannot store through one; here the demand is met by the merged chunk, since either piece alone would have been too little. Third, the value is replaced: the chunk keeps its location and its amount, its value becomes the e-class of the term written, and no other chunk is examined at all. The fresh value created by the first add is replaced without ever having been read, which is the usual outcome for a value inhaled and then overwritten.

The obligation the second step raises is

$$p_{\text{held}} \geq b$$

under the instruction's path condition, and it is an obligation like any other: what discharges it is the route of Section 4.3, with nothing special to the heap about it. It is an inequality rather than an equality because an amount above the bound leaves the state inconsistent by the axiom of Section 4.4, so letting the write through costs nothing.

The bound is what makes the check mean exclusivity, so a location of a kind bounded by `*` can never be written to: with no b, no amount rules out a second holder of the same location, and exclusivity is unprovable in principle rather than merely unproven here. Nothing is lost by it, since the kinds left unbounded are the ones nothing writes through directly — predicate instances (*Predicates*), which are mutated by unfolding them and writing to the fields inside.

A write is not primitive in the way the others are. It could be desugared to a subtract of b, an add of b — which creates a fresh value, the subtract having dropped the chunk — and an assumption equating that value with the one written. That means the same thing, but it tears a chunk down and rebuilds it, introduces an e-class nothing needs, and leaves an equality for congruence closure to merge, where `assign` replaces the value in place after a single lookup. Mutation is everywhere, so the difference is paid on the hot path.

Reading. A read is the first instruction here that needs a value out of a heap. It is *heap-dependent*: it produces an ordinary value, consults a heap without producing one, and names the heap it reads in brackets, `*[h] e` being the value stored at location `e` in `h`. A heap on the left means a state was produced, a heap in brackets means one was consulted.

Viper	VMIR-lite

`assert x.f > 10`	`e0: Int := *[h2] f(x)`
	`e1: Bool := e0 > 10`
	`assert e1`

Listing 12: A read names the heap it consults and yields an ordinary value.

The location is resolved as it is for a write, but the obligation is only *positivity*,

$$p_{\text{held}} > 0$$

again under the path condition: any amount above zero entitles the program to look, and where no chunk answers the amount held is zero, so reading without permission is caught by the same check.

Framing is then a matter of which temporary the instruction mentions. The read of Listing 12 names `h2`, which is by construction what the assignment produced, so it reaches what was just written with no framing argument to make.

Giving permission back. An `exhale` subtracts what the assertion names, with the fraction carried across unchanged:

Viper	VMIR-lite
`exhale acc(x.f, 1/2)`	`h3, _ := h2 - f(x) @ 1/2`

Listing 13: An `exhale` subtracts the amount the assertion names.

A subtract has a second result, which is why Listing 13 binds a pair. It is the counterpart of the bind point, and it points the other way: an add is told what value to put somewhere, a subtract *discovers* what value was there and hands it back. Nothing in Listing 13 wants it, so the binder is `_` and the value is never built. The one construct that does want it is an unfold (*Predicates*), which passes it straight into the instruction that reproduces the footprint, and that is the whole reason the yield exists.

What is handed back is an `Option`, `Some(v)` exactly where the amount taken away is positive, for the same reason a snapshot slot is (*Predicates*): a subtract of no permission removes nothing and so has nothing to report. This is the only position in the instruction set where a value is optional, and the asymmetry is deliberate — presence is *discovered* here, whereas a bind point *supplies* a value and the instruction works out for itself, from the amount, whether a chunk results.

An underscore is written by the translator, never derived. Whether a result is wanted is part of what an instruction is, not an accident of what a later one happens to read, and *Functions* has a case where blanking a result changes the instruction's meaning rather than merely its cost.

A subtract can fail in two ways. It carries the same non-negativity side condition on the amount its `acc` names, and it raises one of its own, that enough was held to take that amount away:

$$p_{\text{held}} \geq p_{\text{needed}}$$

with the amount being taken on the right. Where the second cannot be shown the instruction reports insufficient permission rather than producing a heap. What survives an instruction that does go through is the chunk with its amount replaced by the term $p_{\text{held}} - p_{\text{needed}}$, built and left unevaluated; here that is `1/2`, so the chunk stays. The chunk is dropped only where the difference is *provably* zero, and that is an optimisation rather than a matter of soundness: a chunk holding no permission is inert.

References

[1] P. Müller, M. Schwerhoff, and A. J. Summers, "Viper: A Verification Infrastructure for Permission-Based Reasoning," in *Proceedings of the 17th International Conference on Verification, Model Checking, and Abstract Interpretation - Volume 9583*, in VMCAI 2016. St. Petersburg, FL, USA: Springer-Verlag, 2016, pp. 41–62. doi: 10.1007/978-3-662-49122-5_2.

[2] Programming Methodology Group, ETH Zurich, "Silver: The Viper Intermediate Verification Language."

[3] M. Schwerhoff, "Advancing Automated, Permission-Based Program Verification Using Symbolic Execution," PhD thesis, Zurich, Switzerland, 2016.